The keyspace of the session can be changed using *set_keyspace()* or with the execution of USE *<keyspacename>* as follows:

```
session.set_keyspace('MyKeySpace')
session.execute('USE 'MyKeySpace')
```

Execution of queries

The simplest method for the execution of a query is to call the *execute()* method, as follows:

```
rows = session.execute('SELECT name, age, email FROM
MyKeySpace')
for myrecord in rows:
    print myrecord.myname, myrecord.myage, myrecord.myemail
```

Programming Python - MongoDB

MongoDB is another excellent and cross-platform NoSQL database. It can be classified as a document-oriented database. The latest release of MongoDB is 3.2 for supercomputing mission-critical deployment applications.

The key features of MongoDB include:

- Replication for high availability
- Load balancing
- MapReduce for aggregation functions
- JavaScript at the server side

The users of MongoDB include Adobe, McAfee, SAP, eBay, Craigslist, LinkedIn and many other corporate giants.

First, MongoDB and PyMongo are installed so that the connection of Python with MongoDB can be established.

In a Python shell, the following instruction is executed:

```
$ import pymongo
```

After implementing this step, MongoClient is created by running the *mongod* instance.

The following code is used to connect the default host and port on Python:

```
>>> from pymongo import MongoClient
>>> myMongoclient = MongoClient()
```

The specific host and port can be mentioned explicitly as:

```
>>> myMongoclient = MongoClient('localhost', 27017)
```

Figure 2: MongoDB NoSQL database

Alternatively, the MongoDB URI format can be used as follows:

```
>>> myMongoclient = MongoClient('mongodb://localhost:27017')
```

The MongoDB instance is able to support multiple and independent databases. The databases can be accessed with the use of attribute style access on MongoClient instances, as follows:

```
>>> myNoSQLdb = myclient.MyTestDatabase
```

Representation of documents

The data in MongoDB is stored and represented JSON-style. Using PyMongo, dictionaries can be used for the representation of documents.

The following dictionary is used to represent the blog post:

```
>>> import datetime
>>> post = {"author": "BlogAuthorName",
...         "text": "My Message",
...         "tags": ["NoSQL", "MongoDB", "BigData"],
...         "date": datetime.datetime.utcnow()}
```

Inserting a document

To insert a document into a Cassandra collection, the *insert_one()* method can be used, as follows:

```
>>> myposts = myNoSQLdb.posts
>>> mypostid = myposts.insert_one(mypost).inserted_id
>>> mypostid
```

After the insertion of the document, the collection of *posts* is created on the server. It can be verified by listing all the collections in the database:

```
>>> myNoSQLdb.collection_names(include_system_collections=False)
```

Fetching a document

To fetch a document, type:

```
>>> myposts.find_one()
{u'date': datetime.datetime(...), u'text': u'My Message',
u'_id': ObjectId('ObjectID'), u'author': u'BlogAuthorName',
u'tags': [u'NoSQL', u'MongoDB', u'BigData']}
```

In a similar way, other operations can be implemented for big data processing and analytics. END

By: Dr Gaurav Kumar

The author is the MD of Magma Research and Consultancy Pvt Ltd, Ambala. He is associated with a number of academic institutes, where he delivers lectures and conducts technical workshops on the latest technologies and tools. You can contact him at *kumargaurav.in@gmail.com* or *www.gauravkumarindia.com*.